

IF25-22017 PENGEMBANGAN APLIKASI MOBILE

Modul Belajar 5

Navigasi Antar Layar

Menyambungkan layar dengan NavHost, membawa id ke layar detail, dan membuat menu bawah yang rapi

Muhammad Habib Algifari, S.Kom., M.T.I.

Pendamping Pertemuan 5. Tahun 2026

Komponen	Keterangan
Pertemuan	5
Capaian	CPMK0501, menerapkan konsep pemrograman dalam bentuk aplikasi multi-layar dengan navigasi dan passing data
Waktu belajar	sekitar 5 jam belajar mandiri, ditambah 2,5 jam di kelas

MODUL 5

Peta belajar

Ikuti urutan ini dari atas ke bawah. Setiap bagian memakai apa yang sudah dibahas di bagian sebelumnya.

Urutan	Bagian
1	Tentang modul ini
2	Masalah pembuka: pindah layar dengan if
3	Langkah 1: Tiga bagian navigasi
4	Langkah 2: Menyiapkan dependency dan rute
5	Langkah 3: NavHost untuk LaporanKampus
6	Langkah 4: Back stack, tumpukan riwayat layar
7	Langkah 5: Mengatur tumpukan dengan opsi navigate
8	Langkah 6: Kirim id, bukan seluruh objek
9	Langkah 7: Layar detail memuat datanya sendiri
10	Langkah 8: Dua cara menulis rute
11	Langkah 9: Menu bawah tanpa menumpuk
12	Langkah 10: Memilih pola dan menggambar peta navigasi
13	Latihan mandiri
14	Bawa ke proyekmu
15	Rangkuman
16	Cek pemahaman
17	Bacaan lanjut dan persiapan minggu depan

PEMBUKA

Tentang modul ini

Modul ini menemani Pertemuan 5. Sampai Modul 4, LaporanKampus punya dua layar: daftar dan form. Keduanya masih ditukar dengan `if` biasa. Di modul ini kita mengganti cara itu dengan navigasi sungguhan, menambah layar detail, dan membuat menu bawah dengan tiga tab.

Modul ini terdiri dari tiga bagian. Langkah 1 sampai 5 membahas NavHost dan back stack, yaitu tumpukan riwayat yang dipakai tombol Kembali. Langkah 6 sampai 8 membahas cara mengirim data antar layar. Langkah 9 dan 10 membahas menu bawah dan peta navigasi lima layar.

Setelah menyelesaikan modul ini, kalian bisa

No	Kemampuan	Dibahas di
1	Menyambungkan layar dengan NavHost dan NavController, dan menjelaskan back stack	Langkah 1 sampai 5
2	Membuka layar detail dengan id lewat rute yang aman tipenya	Langkah 6 sampai 8
3	Membuat menu bawah tiga tab yang tidak menumpuk back stack	Langkah 9 dan 10

Capaian ini mendukung **CPMK0501**, dalam bentuk aplikasi multi-layar dengan navigasi dan passing data.

Yang perlu disiapkan

Proyek LaporanKampus dengan layar daftar dari Modul 3 dan form dengan ViewModel dari Modul 4, termasuk objek `PenyimpananLaporan`. Untuk mencoba tombol Kembali, jalankan di emulator Android. Target desktop tidak punya tombol Kembali sistem.

PENTING Mengingat Modul 4

Pola state turun, event naik akan muncul lagi. Layar tidak memanggil navigasi sendiri. Ia hanya mengirim event seperti `bukaDetail(7)` ke atas, dan yang memutuskan pindah ke mana adalah `App.kt`.

PEMBUKA

Masalah pembuka: pindah layar dengan if

Banyak kelompok menyambungkan layar minggu lalu dengan cara seperti ini:

commonMain/App.kt

```
@Composable
fun LaporkampusApp() {
    var layar by remember { mutableStateOf("daftar") }
    when (layar) {
        "daftar" → LayarDaftar(bukaForm = { layar = "form" })
        "form" → FormLaporanScreen(selesai = { layar = "daftar" })
    }
}
```

Kode ini berjalan. Tetapi ada dua masalah.

TEBAK DULU **Coba pikirkan**

Jalankan di emulator Android, buka form, lalu tekan tombol Kembali di HP. Apa yang terjadi? Lalu, bagaimana caranya membuka detail laporan nomor 7 dengan cara ini?

JAWABAN

Tombol Kembali **menutup aplikasi**, bukan kembali ke daftar. Android tidak tahu bahwa ada riwayat layar, karena perpindahan kita hanya mengganti isi satu variabel. Untuk detail, kita butuh membawa angka 7, padahal variabel `layar` hanya berisi teks nama layar dan tidak punya tempat untuk data. Kedua masalah ini dijawab oleh pustaka **Navigation**.

LANGKAH 1

Tiga bagian navigasi

Navigasi di Compose Multiplatform punya tiga bagian:

Bagian	Artinya	Di Laporkampus
Rute	Alamat sebuah layar	RuteBeranda, RuteDetail(7), RuteForm
NavHost	Peta: daftar semua rute dan layar apa yang digambar untuk tiap rute	Satu NavHost di App.kt

Bagian	Artinya	Di Laporkampus
NavController	Pengemudi: pindah ke rute lain, mengingat riwayatnya, dan kembali	<code>nav.navigate(...)</code> dan <code>nav.popBackStack()</code>

ANALOGI Angkot dan haltenya

Bayangkan angkot di Bandar Lampung. Trayek adalah peta jalur yang dilalui, itulah NavHost. Nama halte adalah alamat tujuan, itulah rute. Sopir adalah NavController. Penumpang cukup bilang turun di halte mana, sopir yang tahu jalannya. Sopir juga ingat halte-halte yang sudah dilewati, dan catatan itulah yang disebut back stack.

LANGKAH 2

Menyiapkan dependency dan rute

Dependency

Navigasi untuk Compose Multiplatform disediakan JetBrains. Daftarkan di `gradle/libs.versions.toml`, lalu pakai di `commonMain`.

`gradle/libs.versions.toml`

```
[libraries]
navigation-compose = { module = "org.jetbrains.androidx.navigation:navigation-compose",
version.ref = "navigation" }

[plugins]
kotlinSerialization = { id = "org.jetbrains.kotlin.plugin.serialization", version.ref =
"kotlin" }
```

`composeApp/build.gradle.kts`

```
plugins {
    alias(libs.plugins.kotlinSerialization)
}

kotlin {
    sourceSets {
        commonMain.dependencies {
            implementation(libs.navigation.compose)
        }
    }
}
```

Isi nomor versi `navigation` di bagian `[versions]` sesuai dokumentasi Kotlin Multiplatform yang berlaku saat proyek dibuat. Proyek hands-on P5 di repo memakai versi lama dari seri 2.8. Untuk

proyek kelompok, pakai versi terbaru yang tercantum di dokumentasi dan samakan di semua anggota.

Rute bertipe

commonMain/nav/Rute.kt

```
import kotlinx.serialization.Serializable

@Serializable object RuteBeranda
@Serializable data class RuteDetail(val id: Int)
@Serializable object RuteForm
@Serializable object RuteAsisten
@Serializable object RuteProfil
```

Rute tanpa data cukup ditulis sebagai `object`. Rute yang membawa data memakai `data class`, sehingga tipe argumennya diperiksa compiler. Anotasi `@Serializable` dibutuhkan supaya rute bisa diubah menjadi bentuk yang disimpan pustaka navigasi, dan itulah alasan kita memasang plugin serialization.

LANGKAH 3

NavHost untuk Laporkampus

Sebelum menulis NavHost, kita siapkan dulu layar daftar yang menerima dua lambda: satu untuk membuka detail, satu untuk membuka form.

commonMain/ui/LayarDaftar.kt

```
@Composable
fun LayarDaftar(bukaDetail: (Int) → Unit, bukaForm: () → Unit) {
    val daftar by PenyimpananLaporan.daftar.collectAsState()
    Scaffold(
        floatingActionButton = {
            FloatingActionButton(onClick = bukaForm) {
                Icon(Icons.Default.Add, contentDescription = "Buat Laporan")
            }
        }
    ) { padding →
        Box(Modifier.padding(padding)) {
            DaftarLaporan(daftar, onPilih = { bukaDetail(it.id) })
        }
    }
}
```

Sekarang NavHost-nya:

commonMain/App.kt

```

import androidx.navigation.compose.NavHost
import androidx.navigation.compose.composable
import androidx.navigation.compose.rememberNavController
import androidx.navigation.toRoute

@Composable
fun LaporanKampusApp() {
    val nav = rememberNavController()
    NavHost(navController = nav, startDestination = RuteBeranda) {
        composable<RuteBeranda> {
            LayarDaftar(
                bukaDetail = { id → nav.navigate(RuteDetail(id)) },
                bukaForm = { nav.navigate(RuteForm) }
            )
        }
        composable<RuteDetail> { entry →
            val rute: RuteDetail = entry.toRoute()
            LayarDetail(id = rute.id, kembali = { nav.popBackStack() })
        }
        composable<RuteForm> {
            FormLaporanScreen(selesai = { nav.popBackStack() })
        }
    }
}

```

Bagian	Artinya
<code>rememberNavController()</code>	Membuat pengemudi yang bertahan saat composable dipanggil ulang
<code>startDestination = RuteBeranda</code>	Layar pertama yang dibuka
<code>composable<RuteDetail> { }</code>	Saat rute ini dituju, gambar layar di dalam blok
<code>entry.toRoute()</code>	Membaca kembali objek rute beserta datanya, di sini id
<code>nav.navigate(...)</code>	Pindah ke rute lain
<code>nav.popBackStack()</code>	Kembali ke layar sebelumnya

PENTING Layar tidak kenal NavController

Perhatikan bahwa LayarDaftar tidak menerima nav. Ia hanya menerima lambda bukaDetail dan bukaForm, sama seperti event naik di Modul 4. Dengan begitu, layar bisa dicoba dan diuji tanpa navigasi. Kalau suatu saat rute berubah, cukup App.kt yang diubah.

LayarDetail akan kita isi lengkap di Langkah 7. Untuk sementara, cukup tampilkan id-nya supaya aplikasi bisa dijalankan:

```
commonMain/ui/LayarDetail.kt
```

```
@Composable
fun LayerDetail(id: Int, kembali: () → Unit) {
    Column(Modifier.padding(16.dp)) {
        TextButton(onClick = kembali) { Text("Kembali") }
        Text("Detail laporan #$id", fontSize = 24.sp)
    }
}
```

Kembali otomatis setelah form terkirim

FormLaporanScreen di Modul 4 belum punya parameter `selesai`. Tambahkan, lalu panggil setelah laporan terkirim:

```
commonMain/ui/FormLaporanScreen.kt

@Composable
fun FormLaporanScreen(
    selesai: () → Unit,
    vm: FormLaporanViewModel = viewModel { FormLaporanViewModel() }
) {
    val state by vm.uiState.collectAsState()
    LaunchedEffect(state.pesan) {
        if (state.pesan != null) selesai()
    }
    // isi form sama seperti Modul 4
}
```

`LaunchedEffect` menjalankan blok di dalamnya sekali, lalu sekali lagi setiap kali nilai kuncinya berubah. Di sini kuncinya `state.pesan`. Begitu pesan Laporan terkirim muncul, kita kembali ke daftar. Ingat aturan di Modul 4: efek samping tidak boleh ditulis langsung di badan composable. `LaunchedEffect` adalah tempat yang aman untuk itu.

LANGKAH 4

Back stack, tumpukan riwayat layar

Setiap kali `navigate` dipanggil, layar baru ditaruh di atas tumpukan. Setiap kali Kembali ditekan atau `popBackStack` dipanggil, layar paling atas diambil. Tumpukan ini disebut **back stack**.

ANALOGI Tumpukan piring

Back stack seperti tumpukan piring di wastafel. `navigate` menaruh piring baru di paling atas. Kembali mengambil piring paling atas. Piring yang paling bawah, yaitu layar pertama, diambil terakhir. Kalau tumpukannya habis, tidak ada lagi yang bisa dicuci, dan aplikasi ditutup.

TEBAK DULU

Kita membuka Daftar, lalu Detail 7, lalu Form. Setelah itu tombol Kembali ditekan dua kali. Layar apa yang tampil? Apa yang terjadi kalau Kembali ditekan sekali lagi?

JAWABAN

Isi tumpukan berubah seperti ini:

Kejadian	Isi back stack, dari bawah ke atas	Yang tampil
Buka aplikasi	Daftar	Daftar
Ketuk kartu 7	Daftar, Detail 7	Detail 7
Tekan ubah	Daftar, Detail 7, Form	Form
Kembali	Daftar, Detail 7	Detail 7
Kembali	Daftar	Daftar
Kembali	kosong	Aplikasi ditutup

Jadi setelah dua kali Kembali, yang tampil adalah **Daftar**. Satu kali lagi menutup aplikasi, karena tumpukan sudah kosong.

LANGKAH 5

Mengatur tumpukan dengan opsi navigate

Kadang kita tidak ingin layar tertentu tetap ada di tumpukan. Contohnya, setelah laporan terkirim, pengguna tidak boleh bisa kembali ke form yang sudah terkirim.

commonMain/App.kt

```
// setelah laporan terkirim: buka detailnya,
// dan buang Form dari tumpukan
nav.navigate(RuteDetail(idBaru)) {
    popUpTo<RuteBeranda>()
    launchSingleTop = true
}
```

Opsi	Gunanya
<code>popUpTo<Rute>()</code>	Buang semua layar di atas rute itu sebelum pindah
<code>inclusive = true</code>	Ikut buang rute itu sendiri
<code>launchSingleTop = true</code>	Jangan buat dubel kalau layar yang sama sudah di paling atas

Opsi	Gunanya
saveState, restoreState	Simpan dan pulihkan isi tab, dipakai di Langkah 9

Dengan kode di atas, isi tumpukan setelah kirim adalah Daftar, lalu Detail laporan baru. Menekan Kembali dari detail langsung ke daftar, tidak melewati form.

CONTOH Contoh sehari-hari

Kasus lain yang sering ditemui adalah layar login. Setelah berhasil masuk, layar login dibuang dengan `popUpTo<RuteLogin> { inclusive = true }`. Kalau tidak dibuang, pengguna yang menekan Kembali di beranda justru kembali ke layar login, padahal sudah masuk. Ini mirip loket tiket: setelah tiket diterima, kalian tidak kembali mengantre di loket yang sama.

LANGKAH 6

Kirim id, bukan seluruh objek

Saat membuka detail, kita mengirim id laporan, bukan seluruh objek Laporan. Bandingkan:

	Kirim id	Kirim objek Laporan
Ukuran	Satu angka	Seluruh isi, termasuk foto
Kesegaran data	Detail mengambil versi terbaru	Bisa basi kalau status berubah setelah dikirim
Proses dimatikan sistem	Rute bisa dipulihkan	Objek besar bisa hilang
Tautan dari luar	Alamat seperti detail/7 mudah dibuat	Tidak mungkin

Aturan praktisnya: **argumen navigasi berisi kunci, bukan data**. Layar tujuan yang memuat datanya sendiri lewat ViewModel.

ANALOGI Nomor resi paket

Saat menanyakan paket ke kurir, kalian cukup menyebut nomor resi, bukan membawa paketnya ke kantor. Dengan nomor resi, kurir bisa melihat status terbaru kapan saja. Kalau yang kalian bawa adalah foto paket kemarin, informasinya bisa sudah basi.

LANGKAH 7

Layar detail memuat datanya sendiri

Layar detail punya tiga kemungkinan keadaan: data sedang dimuat, data ada, atau laporan dengan id itu tidak ditemukan. Ingat sealed class di Modul 2? Ini tempat yang pas untuk memakainya.

commonMain/ui/DetailViewModel.kt

```
sealed class DetailUiState {
    data object Memuat : DetailUiState()
    data class Ada(val laporan: Laporan) : DetailUiState()
    data object TidakDitemukan : DetailUiState()
}

class DetailViewModel(id: Int) : ViewModel() {
    val uiState: StateFlow<DetailUiState> = PenyimpananLaporan.daftar
        .map { semua →
            val laporan = semua.find { it.id == id }
            if (laporan == null) DetailUiState.TidakDitemukan
            else DetailUiState.Ada(laporan)
        }
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000),
            DetailUiState.Memuat)
}
```

Bagian	Artinya
PenyimpananLaporan.daftar	StateFlow berisi semua laporan, dari Modul 4
.map { }	Operator Flow dari Modul 2, mengubah daftar menjadi keadaan detail
semua.find { }	Mencari laporan dengan id yang cocok, hasilnya null kalau tidak ada
.stateIn(...)	Mengubah Flow menjadi StateFlow dengan nilai awal Memuat
WhileSubscribed(5000)	Aliran tetap hidup 5 detik setelah layar berhenti mengamati, supaya tidak dimulai ulang saat layar diputar

Karena detail membaca dari PenyimpananLaporan, kalau status laporan berubah di tempat lain, layar detail ikut berubah. Inilah keuntungan mengirim id di Langkah 6.

PENTING Versi di slide

Di slide, sumber datanya adalah `sumber.pantau(id)` dari interface `Sumber` di Modul 2. Polanya sama persis: Flow dari sumber data, diubah menjadi StateFlow dengan `stateIn`. Pakai yang sesuai dengan kode kelompok kalian.

Lalu sambungkan di NavHost dan tampilkan ketiga keadaannya:

commonMain/App.kt

```
composable<RouteDetail> { entry →
    val id = entry.toRoute<RouteDetail>().id
    val vm = viewModel { DetailViewModel(id) }
    val state by vm.uiState.collectAsState()
    LayarDetail(state, kembali = { nav.popBackStack() })
}
```

commonMain/ui/LayarDetail.kt

```
@Composable
fun LayarDetail(state: DetailUiState, kembali: () → Unit) {
    Column(Modifier.padding(16.dp), verticalArrangement = Arrangement.spacedBy(8.dp)) {
        TextButton(onClick = kembali) { Text("Kembali") }
        when (state) {
            DetailUiState.Memuat → CircularProgressIndicator()
            DetailUiState.TidakDitemukan → Text("Laporan tidak ditemukan")
            is DetailUiState.Ada → {
                Text(state.laporan.judul, fontSize = 24.sp, fontWeight =
                FontWeight.Bold)
                Text(state.laporan.lokasi, color = Color.Gray)
                LabelStatus(state.laporan.status)
                Text(state.laporan.status.label())
            }
        }
    }
}
```

LabelStatus dari Modul 3 dan `label()` dari Modul 2 dipakai lagi di sini tanpa diubah.

LANGKAH 8

Dua cara menulis rute

Selain rute bertipe, ada cara lama yang menulis rute sebagai teks. Cara ini masih banyak di tutorial, jawaban AI, dan hands-on P5 di repo.

	Rute bertipe	Rute string
Mendaftarkan	<code>composable<RouteDetail></code>	<code>composable("detail/{itemId}", arguments = ...)</code>
Pindah	<code>nav.navigate(RouteDetail(7))</code>	<code>nav.navigate("detail/7")</code>
Membaca	<code>entry.toRoute<RouteDetail>().id</code>	<code>entry.arguments?.getInt("itemId")</code>
Salah ketik	Ketahuan saat compile	Baru ketahuan saat aplikasi crash

Konsepnya sama persis, hanya alamatnya yang ditulis sebagai teks. Kenali keduanya. Untuk proyek kelompok kita pakai rute bertipe.

ANALOGI Kontak di HP

Rute bertipe seperti memilih nama dari daftar kontak: tidak mungkin salah nomor. Rute string seperti mengetik nomor HP secara manual setiap kali menelepon: satu angka salah, teleponnya nyasar, dan kalian baru tahu saat tersambung ke orang lain.

TEBAK DULU

Rute terdaftar sebagai `detail/{itemId}` dengan tipe `Int`. Lalu kita memanggil `nav.navigate("detail/abc")`. Apa yang terjadi? Bagaimana dengan `nav.navigate(RuteDetail("abc"))`?

JAWABAN

Yang pertama bisa di-build, tetapi aplikasi **crash saat tombol ditekan** dengan `IllegalArgumentException`, karena tidak ada rute yang cocok. Yang kedua **tidak bisa di-build sama sekali**, karena id bertipe `Int`. Inilah alasan kita memilih rute bertipe: kesalahan pindah ke saat compile, sebelum aplikasi sampai ke tangan pengguna.

LANGKAH 9

Menu bawah tanpa menumpuk

NavigationBar di dalam Scaffold

Tiga tujuan utama `LaporKampus`, yaitu `Beranda`, `Asisten`, dan `Profil`, kita taruh di menu bawah. Pertama, daftar tabnya:

`commonMain/nav/Tab.kt`

```
data class Tab(val label: String, val ikon: ImageVector, val rute: Any)

val TAB = listOf(
    Tab("Beranda", Icons.Default.Home, RuteBeranda),
    Tab("Asisten", Icons.Default.Face, RuteAsisten),
    Tab("Profil", Icons.Default.Person, RuteProfil)
)
```

Lalu bungkus `NavHost` dengan `Scaffold` yang punya `bottomBar`:

`commonMain/App.kt`

```

@Composable
fun LaporanKampusApp() {
    val nav = rememberNavController()
    Scaffold(bottomBar = {
        NavigationBar {
            val entry by nav.currentBackStackEntryAsState()
            TAB.forEach { tab →
                NavigationBarItem(
                    selected = entry?.destination?.hasRoute(tab.rute::class) == true,
                    onClick = { pindahTab(nav, tab.rute) },
                    icon = { Icon(tab.ikon, contentDescription = null) },
                    label = { Text(tab.label) }
                )
            }
        }
    }) { padding →
        NavHost(nav, startDestination = RuteBeranda, modifier =
        Modifier.padding(padding)) {
            // composable<RuteBeranda>, RuteDetail, RuteForm dari Langkah 3 dan 7
            composable<RuteAsisten> { Text("Asisten AI, diisi di Modul 9") }
            composable<RuteProfil> { Text("Profil") }
        }
    }
}

```

`currentBackStackEntryAsState()` membuat menu tahu layar mana yang sedang aktif, sehingga tab yang benar tampil terpilih. `hasRoute(...)` memeriksa apakah layar itu milik rute tab tersebut.

Pindah tab tanpa menumpuk

Kalau `onClick` hanya memanggil `nav.navigate(tab.rute)`, setiap ketukan tab menambah satu layar ke tumpukan. Setelah bolak-balik antartab, pengguna harus menekan Kembali berkali-kali untuk keluar.

commonMain/nav/PindahTab.kt

```

import androidx.navigation.NavController
import androidx.navigation.NavGraph.Companion.findStartDestination

fun pindahTab(nav: NavController, rute: Any) {
    nav.navigate(rute) {
        popUpTo(nav.graph.findStartDestination().id) {
            saveState = true
        }
        launchSingleTop = true
        restoreState = true
    }
}

```

Tiga opsi ini selalu dipasang bersama untuk menu bawah:

Opsi	Gunanya
<code>popUpTo(awal) { saveState = true }</code>	Buang layar di atas Beranda, tetapi simpan isinya

Opsi	Gunanya
<code>launchSingleTop = true</code>	Mengetuk tab yang sedang aktif tidak membuat salinan baru
<code>restoreState = true</code>	Saat tab dibuka lagi, pulihkan isi dan posisi gulirnya

`saveState` dan `restoreState` berpasangan. Yang satu menyimpan posisi gulir dan isi tab saat ditinggal, yang lain memulihkannya saat kembali.

ANALOGI Pembatas buku

Bayangkan kalian membaca tiga buku bergantian. `saveState` seperti menyelipkan pembatas di halaman terakhir yang dibaca sebelum menutup buku. `restoreState` seperti membuka buku tepat di halaman pembatas. Tanpa keduanya, setiap buku dibuka lagi dari halaman pertama.

TEBAK DULU

Tanpa opsi apa pun, pengguna mengetuk tab Asisten, Profil, Asisten, lalu Profil. Berapa kali Kembali harus ditekan sampai aplikasi tertutup? Bagaimana kalau memakai `pindahTab`?

JAWABAN

Tanpa opsi, tumpukannya berisi Beranda, Asisten, Profil, Asisten, Profil. Kembali harus ditekan **lima kali**. Dengan `pindahTab`, tumpukan hanya berisi Beranda dan tab yang sedang aktif, yaitu Beranda dan Profil. Kembali cukup ditekan **dua kali**: sekali ke Beranda, sekali lagi menutup aplikasi.

LANGKAH 10

Memilih pola dan menggambar peta navigasi

Menu bawah, drawer, atau tombol kembali

Pola	Kapan dipakai	Contoh di Laporkampus
<code>NavigationBar</code>	Tiga sampai lima tujuan utama yang sering dibuka	Beranda, Asisten, Profil
<code>ModalNavigationDrawer</code>	Banyak tujuan yang jarang dibuka	Pengaturan, bantuan, tentang aplikasi

Pola	Kapan dipakai	Contoh di Laporkampus
Tombol Kembali di bagian atas	Layar turunan dari layar utama	Detail laporan dan form

Drawer adalah menu yang ditarik dari tepi kiri layar. Ia ada di RPS minggu ini, tetapi untuk aplikasi kecil menu bawah lebih mudah ditemukan pengguna. Kelompok boleh memakai drawer kalau tujuannya memang banyak.

Peta navigasi Laporkampus

Rute	Layar	Dibuka dari
RuteBeranda	Daftar laporan	Tab Beranda, layar pertama
RuteAsisten	Fitur AI, diisi di Modul 9	Tab Asisten
RuteProfil	Profil dan pengaturan	Tab Profil
RuteDetail(id)	Detail satu laporan	Ketuk kartu di daftar
RuteForm	Tambah atau ubah laporan	Tombol Lapork di daftar, atau ubah di detail

Lima layar ini sama dengan lima layar wajib di Bank Proyek: daftar, detail, form, fitur AI, dan profil.

PENTING Gambar petanya dulu

Gambar peta seperti ini di README proyek kelompok sebelum menulis kodenya. Kotak untuk setiap layar, panah untuk setiap perpindahan, dan tulis data apa yang dibawa di atas panahnya. Peta ini juga yang akan ditanyakan saat tes lisan 1 di Pertemuan 12: dari layar mana ke layar mana, dan rute apa yang membawa data.

PRAKTIK

Latihan mandiri

Ada tiga latihan di folder P5 Navigasi Antar Layar Hands-on. Semuanya memakai rute string. Latihan 2 dan 3 yang diamati.

Latihan	Berkas	Yang dilatih
1	Handson1NavHostDasar.kt	Dua layar, Home dan Detail, dengan navigate dan popBackStack
2	Handson2PassingData.kt	Ketuk item di daftar untuk membuka detail dengan itemId yang benar
3	Handson3BottomNavigation.kt	Tiga tab dengan bug back stack yang perlu diperbaiki

CONTOH Petunjuk latihan 1

Polanya sama dengan Langkah 3. Layar Home menerima lambda untuk pindah, bukan NavController. Uji di emulator Android: dari Detail, tombol Kembali di HP harus kembali ke Home.

CONTOH Petunjuk latihan 2

Pastikan tiga tempat memakai nama yang sama persis: pola rute `detail/{itemId}`, daftar arguments dengan nama `itemId` bertipe `Int`, dan `getInt("itemId")` saat membaca. Satu huruf beda di salah satu tempat sudah cukup membuat aplikasi crash. Untuk tantangan, ubah latihan ini menjadi rute bertipe seperti Langkah 2 dan 3.

CONTOH Petunjuk latihan 3

Jalankan dulu sebelum memperbaiki. Pindah tab beberapa kali, lalu hitung berapa kali Kembali harus ditekan sampai keluar. Setelah itu, terapkan tiga opsi di Langkah 9, jalankan lagi, dan bandingkan hitungannya. Perhatikan juga apakah posisi gulir sebuah tab tetap sama saat kalian kembali ke tab itu.

Yang diamati

Level	Skor	Yang terlihat
Sangat baik	85 sampai 100	Detail membuka item yang benar lewat itemId dan bug back stack di tab sudah diperbaiki

Level	Skor	Yang terlihat
Baik	70 sampai 84	Passing data berjalan, bug back stack belum diperbaiki
Cukup	55 sampai 69	Dua layar tersambung tanpa argumen
Kurang	di bawah 55	Belum ada navigasi yang berjalan

Observasi dikerjakan individu dengan bobot 4% dari nilai akhir, untuk CPMK0501. Buktinya: push latihan ke repository latihan pribadi, lalu kirim tautan commit di LMS sebelum kelas selesai.

Kalau error

Gejala atau pesan	Penyebab	Yang dicoba
Navigation destination that matches route cannot be found	Rute tidak sama persis atau tipe argumen salah	Samakan string rute, atau pakai rute bertipe
Kembali harus ditekan berkali-kali	navigate tab tanpa popUpTo	Pakai pindahTab dengan popUpTo, saveState, dan launchSingleTop
Isi tab hilang saat kembali	Hanya salah satu dari saveState dan restoreState	Pasang keduanya
Unresolved reference: toRoute atau Serializable	Plugin serialization belum dipasang	Tambahkan plugin kotlin-serialization
Serializer for class not found	Rute lupa diberi @Serializable	Tambahkan anotasi pada setiap rute

PRAKTIK

Bawa ke proyekmu

Sambungkan lima layar tema kelompok kalian. Layar AI dan profil boleh masih berisi teks sementara. Yang penting minggu ini adalah sambungannya. Bagi kerja per layar, tetapi satu orang memegang App.kt supaya tidak bentrok.

- 1. Rute bertipe.** Lima rute sesuai Bank Proyek, dengan rute detail membawa id entitas utama.
- 2. Menu bawah.** Tiga tab tanpa bug back stack, layar lain dibuka lewat tombol dan kartu.
- 3. Detail memuat sendiri.** ViewModel detail mengambil data dari tempat penyimpanan berdasarkan id.
- 4. Peta di README.** Gambar peta navigasi lima layar kelompok kalian.

CONTOH Contoh dari Laporan Kampus

RuteBeranda, RuteAsisten, dan RuteProfil sebagai tab. RuteDetail(id) dan RuteForm dibuka dari daftar. DetailViewModel memakai sealed class DetailUiState untuk tiga keadaan layar.

Checkpoint minggu 5

No	Yang diselesaikan
1	NavHost dengan lima rute bertipe sesuai Bank Proyek
2	Kartu di daftar membuka detail yang memuat datanya sendiri
3	Tiga tab dengan popUpTo, saveState, dan restoreState

Push ke repository kelompok. README memuat peta navigasi dan tangkapan layar kelima layar. Checkpoint tidak masuk nilai, tetapi menjadi bekal tes lisan di pertemuan 12 sampai 16. Lima layar ini adalah bagian pertama dari fitur utama yang dinilai di tes lisan 1.

PENUTUP

Rangkuman

Navigasi punya tiga bagian: **rute** sebagai alamat layar, **NavHost** sebagai peta, dan **NavController** sebagai pengemudi. Rute ditulis sebagai `object` atau `data class` dengan anotasi `@Serializable`.

Setiap `navigate` menaruh layar di atas **back stack**, dan setiap Kembali mengambil layar paling atas. Opsi `popUpTo`, `inclusive`, dan `launchSingleTop` mengatur isi tumpukan, misalnya membuang form setelah terkirim.

Layar tidak menerima NavController. Ia hanya mengirim event seperti `bukaDetail(id)`, dan App.kt yang memutuskan pindah ke mana.

Argumen navigasi berisi **kunci, bukan data**. Layar tujuan memuat datanya sendiri lewat ViewModel, misalnya dengan `map` dan `stateIn`, dan menampilkan keadaan memuat, ada, atau tidak ditemukan.

Rute bertipe memindahkan kesalahan alamat ke saat compile, sedangkan rute string baru ketahuan saat crash.

Menu bawah memakai NavigationBar di dalam Scaffold. Pindah tab selalu memakai tiga opsi bersama: `popUpTo` dengan `saveState`, `launchSingleTop`, dan `restoreState`.

PENUTUP

Cek pemahaman

Jawab tanpa melihat modul dulu, lalu cocokkan dengan kunci di bawahnya.

1. Kenapa perpindahan layar dengan `if` membuat tombol Kembali menutup aplikasi?
2. Apa isi back stack setelah Daftar, Detail 7, lalu Form, dan apa yang diambil tombol Kembali?
3. Setelah login berhasil, opsi navigate apa yang dipakai supaya Kembali tidak membuka layar login lagi?
4. Kenapa yang dikirim ke layar detail adalah id, bukan objek Laporan?
5. Kenapa LayarDaftar menerima lambda `bukaDetail`, bukan NavController?
6. Apa bedanya rute string dan rute bertipe saat terjadi salah ketik?
7. Apa akibatnya kalau pindah tab tanpa `popUpTo`, dan apa akibatnya kalau hanya memasang `saveState` tanpa `restoreState`?

JAWABAN Kunci jawaban

1. Android tidak tahu ada riwayat layar. Perpindahan hanya mengganti isi satu variabel, jadi tidak ada back stack yang bisa diambil.
2. Daftar, Detail 7, Form. Kembali mengambil Form, sehingga Detail 7 tampil.
3. `popUpTo<RuteLogin> { inclusive = true }`, supaya layar login ikut dibuang dari tumpukan.
4. id kecil, data di detail selalu versi terbaru, dan rute bisa dipulihkan kalau proses aplikasi dimatikan sistem.
5. Supaya layar bisa dicoba dan diuji tanpa navigasi, dan kalau rute berubah cukup `App.kt` yang diubah. Ini pola event naik dari Modul 4.
6. Rute string baru ketahuan saat aplikasi crash, sedangkan rute bertipe sudah ketahuan saat compile.
7. Tanpa `popUpTo`, tumpukan terus bertambah sehingga Kembali harus ditekan berkali-kali. Tanpa `restoreState`, isi tab yang sudah disimpan tidak pernah dipulihkan, sehingga tab selalu kembali ke awal.

PENUTUP

Bacaan lanjut dan persiapan minggu depan

Sumber	Untuk apa
kotlinlang.org, Navigation in Compose Multiplatform	Dependency, rute bertipe, dan contoh kode
developer.android.com, Navigation with Compose	Konsep NavHost dan NavController
developer.android.com, Type safety in Kotlin DSL and Navigation Compose	Rute bertipe dan toRoute
developer.android.com, Navigation bar	Pola menu bawah Material 3
developer.android.com, Side-effects in Compose	LaunchedEffect dan efek samping lainnya

CONTOH Masalah Pertemuan 6

Semua laporan kita masih palsu dan hilang saat aplikasi ditutup. Bagaimana mengambil laporan dari server, menampilkan Memuat, dan memberi pesan yang jelas saat internet kampus mati? Minggu depan sumber data palsu diganti sumber yang memanggil server dengan Ktor, tanpa mengubah layar sama sekali. Pastikan lima layar kelompok sudah tersambung.